

Q1/ Which dependencies did you use and why?

In this project, the primary external dependency used is `http:1.2.0`, which is the official HTTP client package maintained by the Dart team. This package was chosen because it provides a simple and transparent way to make HTTP requests such as `GET`, `POST`, `PUT` and `DELETE` to the JSONPlaceholder REST API. It supports all the HTTP verbs needed to implement full CRUD operations without adding unnecessary complexity or abstraction. Additionally, two built-in Dart libraries were used: `dart:convert`, which provides the `jsonEncode()` and `jsonDecode()` functions used to serialize Dart objects into JSON before sending them to the API and deserialize JSON responses back into Dart objects. The second built-in library is `dart:io`, which was used specifically to catch `SocketException`, the error Dart throws when the device has no internet connection or cannot reach the server. These built-in libraries require no installation since they come with the Dart SDK. Together, these three libraries cover all networking needs of the app: sending requests, parsing responses, and handling connection errors gracefully.

Q2) Handle API exceptions (network error, Invalid data)

1) Try-Catch blocks: This is used to prevent your app from an error occurs. you wrap your API call inside try and catch errors in catch.

if something fails, the program continues running instead of stopping.

2) Network errors: These happens when there is no internet connection or the network is unstable. the app cannot reach the server, so the request fails. you should detect this and show a message like "check your internet." or retry option

3) Timeout Errors: These occur when the server takes too long to respond. Instead of waiting forever, you set a time limit for the request. if the limit is exceeded, the request is stopped.

4) Server Errors (HTTP status codes): These errors come from the server, like 404 (not found) or 500 (server failure).

you check the response status code after the API call. different codes tell you what went wrong.

5) Invalid Data handling: Sometimes the API returns incorrect or unexpected data. for example, the JSON format may be wrong or missing fields.

you should validate and safely parse the data.

6. User-Friendly Error Messages: instead of showing technical errors, display simple messages. Users don't understand system errors like "SocketException". Use messages like "Something went wrong." or "Try again."

7. Centralized error handling: instead of writing error handling everywhere, use one common function. This function processes all types of errors in one place. And it keeps your code clean and easy to maintain.

8. Loading, Success and Error States: your app should show different states during API calls. Loading shows a spinner while waiting for data. Success displays the data and error shows message if it fails. This makes the app feel responsive and clear.

Q3 | Explanation of FutureBuilder Widget

The `FutureBuilder<T>` widget in flutter is used to handle asynchronous operations (like API calls) and automatically update the UI when the result is available.

It listens to a future `<T>` and rebuilds the UI depending on the current state of that future.

Three main states used:

- * **Loading State** (`ConnectionState.waiting`): ~~It shows~~ This state happens when the future is still running (data has not arrived yet).
 - * **Error State** (`Snapshot.hasError`): If something goes wrong (example: network error or server failure) this state is triggered.
 - * **Success State** (`Snapshot.hasData`): This state means the data has been successfully received. The UI now displays the actual content.
- The part of UI depends on FutureBuilder

In `PostListScreen`: The `FutureBuilder` is connected to a future `<List<Post>>`, which fetches a list of posts. Here the entire `Listview` of post cards depends on the `FutureBuilder`.

This means:

- While loading → Shows Spinner
- On error → Shows error message
- On Success → Shows full list of posts (the real contents)

So, the whole screen content changes based on the future's state.